SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Efficient Object Store Retrieval for Analytical Query Processing

Tomás Gouveia Correia de Morais Soares

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Efficient Object Store Retrieval for Analytical Query Processing

Effizientes Abrufen aus Object Stores für die analytische Abfrageverarbeitung

Author: Tomás Gouveia Correia de Morais Soares
Examiner: Viktor Leis
Supervisor: Till Steinert
Submission Date: 05.03.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 05.03.2026 Tomás Gouveia Correia de Morais Soares

Abstract

Modern analytical workloads have shifted toward cloud-native architectures that decouple storage from compute, enabling independent scalability. While this provides significant operational flexibility, the latency introduced by remote storage such as Amazon S3[1] poses a significant challenge for high-performance query engines. This thesis presents the integration of remote storage retrieval into Plan-to-C (P2C), a high-performance query engine written in C++ and developed at the Chair of Decentralized Information Systems and Data Management (DIS) at the Technical University of Munich (TUM).

While P2C achieves high efficiency by leveraging LLVM [2] for Just-In-Time (JIT) compilation, its initial design was restricted to in-memory data sources. Our work bridges that gap by implementing a remote data-fetching layer designed to query Parquet [3] data directly from S3. Following the principles of the AnyBlob framework [4], the implementation utilizes `io_uring`-based asynchronous I/O to minimize kernel overhead during network interaction.

We implement a dynamic thread scheduling strategy based on epoch-based statistics aggregation to balance CPU-intensive processing with I/O-heavy retrieval. Furthermore, we evaluate the system's ability to mask network latency through effective pipelining and adaptive resource allocation. The obtained results demonstrate that the engine can maintain high throughput and CPU utilization under fluctuating network conditions, effectively transitioning P2C from a local to a cloud-integrated analytical engine prototype.

Contents

Abstract **iii**

1 Introduction **1**
1.1 Cloud-Native Data Management 1
1.2 The Challenge of High-Performance Retrieval 1
1.3 Goals of this Thesis 2

2 Background **3**
2.1 LLVM JIT Compilation Logic 3
2.2 Original Data Format 4
2.3 Necessary LLVM Adaptations 4
2.3.1 Table Schema 4
2.3.2 Variable-Sized Datatypes (Strings) 4
2.4 Parquet 5
2.4.1 Row-Group-Based Storage 5
2.4.2 Reading Data from Remote 7
2.5 Chunk Processing Implementation 8
2.5.1 Database and Tables 8
2.5.2 Chunks 8

3 Scheduling Implementation **10**
3.1 Synchronous Baseline 10
3.2 Adaptive Asynchronous Scheduler 11
3.2.1 Thread Jobs 11
3.2.2 Statistics Aggregation 13
3.2.3 Performance Adaptivity 13
3.2.4 Stabilization of Adaptivity Algorithm 14
3.2.5 Scheduling Parameters 16
3.3 Optimization 17
3.3.1 BlockPool 17
3.3.2 Lock-Free Data Structures 18

4 Performance Evaluation **19**
4.1 Parameter Search 19
4.2 Benchmarking Results 21
4.2.1 Irrelevant Parameters 21
4.2.2 Runtime Averages and Variance 22
4.2.3 Final Choice 24
4.2.4 Scheduler Profiling 26
4.2.5 DuckDB Comparison 31

5 Conclusion & Future Work **33**
5.1 Conclusion 33
5.2 Future Work 33

Abbreviations **35**

List of Figures **36**

List of Tables **37**

Bibliography **38**

1 Introduction

1.1 Cloud-Native Data Management

The landscape of analytical data management has undergone a fundamental transformation in the last decade. Historically, high-performance database systems followed a "shared-nothing" architecture, where data was stored on local disks attached to computing nodes. The tight coupling of storage and compute required an in-tandem scaling model, which often led to over-provisioning and significant operational costs. Modern cloud-native architectures have moved away from this model in favor of disaggregated storage, allowing for independent scalability; in this paradigm, data is stored in remote object stores such as Amazon S3 [1], Microsoft Azure [5] or Google Cloud Storage [6], while compute resources are provisioned on-demand. A customer may store petabytes of data at a low cost in a "Data Lake" while only spinning up a large compute cluster for the duration of an individual analytical workflow.

The "multi-cluster shared-data" architecture was most notably pioneered by Snowflake[7]. By decoupling storage from compute, Snowflake demonstrated that the elasticity of the cloud could be leveraged to provide near-infinite scaling without the traditional inherent data-shuffling overhead [8]. Other pioneers, such as Amazon Redshift [9] and Google BigQuery [10], further validated this model, proving that the remote object store — despite its latency — could serve as the primary persistent storage layer for massive analytical workloads.

1.2 The Challenge of High-Performance Retrieval

While local Non-Volatile Memory Express (NVMe) storage provides microsecond latency, fetching data from a remote object store involves network round-trips and software stack overhead that can easily stall a high-performance query engine. We must therefore shift from naive synchronous fetching to adaptive models, where scheduling algorithms ensure the network is treated as a first-class citizen.

The challenge lies in masking the high latency of the cloud by overlapping I/O-retrieval with CPU-heavy processing. The latency of remote storage might remain a challenge, but the physical constraints of the network have shifted dramatically. As noted by Steinert, Kuschewski, and Leis in their Cloudspecs Paper [11], the evolution of cloud infrastructure has led to a paradigm shift in resource availability. Most relevant to our work is the 60x evolution in network bandwidth over the last decade, with modern cloud instances now offering up to 400 Gbit/s or even 600 Gbit/s of throughput. This trajectory suggests that the primary bottleneck could no longer be the physical capacity of the links, but rather the efficiency of the software stack in saturating that bandwidth.

1.3 Goals of this Thesis

The primary objective of this thesis is to transition the P2C Query Engine, developed at the DIS Chair at TUM, from an in-memory prototype to a storage-disaggregated engine capable of asynchronous remote data access. By integrating the principles described in the AnyBlob paper [4], specifically its integration with the Umbra DB System [12] through the use of a specialized scheduler, we aim to demonstrate that a compiled query engine can maintain its performance edge even when data is stored remotely, exploiting the high network bandwidth capabilities of modern cloud instances. We explore the implementation of a Parquet-based [3] storage layer and an adaptive thread-scheduling strategy that dynamically balances the system's resources between network interaction and CPU-side execution.

2 Background

2.1 LLVM JIT Compilation Logic

The P2C engine utilizes the LLVM JIT compiler infrastructure to transform analytical queries into specialized machine code kernels, following the data-centric compilation strategy pioneered by Neumann in his 2011 paper [13]. This approach moves away from the traditional Volcano-style [14] iterator model, which suffers from significant overhead due to virtual function calls and poor CPU cache locality, in favor of the generation of query plans that maximize pipeline efficiency.

The process begins with Pipeline Planning, where queries are divided into sequences of operators that can be executed together without materialization breaks[1]. Each pipeline is represented as a distinct LLVM function, enabling the engine to bypass the overhead of traditional iterative models by processing data in tight, instruction-efficient loops. During `IR` Generation, the `Builder` class abstracts the `llvm::IRBuilder` to create LLVM Intermediate Representation (IR) based on the specific table schema. For example, the Scan operator uses `createColumnAccess` to generate the exact memory offsets required for column retrieval. This allows the JIT compiler to generate a specialized ScanFn kernel that directly manipulates pointers to the raw data provided by the storage layer. Finally, Kernel Execution occurs when the `QueryScheduler` calls the compiled function pointer. The JIT-compiled kernel performs a morsel-driven [15], index-based loop over a data chunk, using a function signature that accepts a pointer array `column_ptrs`.

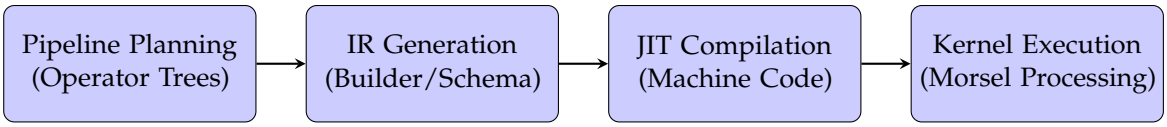


Figure 2.1: LLVM JIT execution flow

[1]Operators such as Joins or Aggregations are pipeline breakers; they must first fully materialize their input into a Hash Table before the subsequent probe/reduction phase is allowed to begin.

2.2 Original Data Format

The efficiency of the JIT-compiled kernels relies on the ability to perform direct pointer arithmetic over memory-resident data, making the physical layout of the underlying storage a critical factor in execution performance. The P2C Query Engine supports a variety of queries for the TPCH benchmark [16] (01, 05, 06, 07, 08, 09, 12, 14, 17, 19). In its original state, the tabular TPCH data was initialized in-memory through structs; the table layout, data types and column names were hardcoded, such that the LLVM pipeline could reference the in-memory (precomputed) addresses of the columns directly, to fetch the data and apply the query kernel function.

For a real-world application, however, arbitrary table structures must be supported. As the Database Schema information is not trivially extracted from the generated column files, we modify the TPCH generation suite to create `json` files containing the Schema metadata alongside the files themselves, more specifically, during the conversion-to-Parquet step (see Section 2.4). The original engine was reliant on a column-based data layout, where the columns were initialized in a contiguous manner in the main memory. This design choice had to be adapted to enable true morsel-driven execution, with overlapping downloading and processing jobs, since a chunk must contain entire rows for processing.

2.3 Necessary LLVM Adaptations

2.3.1 Table Schema

Instead of relying on the hardcoded columnar data, we implement a `SchemaManager` class which loads the schemata from the remote storage on startup for each table from the respective `json` file. For every remotely stored database, this `json` file must be uploaded alongside the data. The table names, column names and column datatypes are stored here, acting as a bridge between the chunk fetching & decompressing logic and the LLVM pipeline execution.

2.3.2 Variable-Sized Datatypes (Strings)

A significant challenge when moving from an in-memory DB to an asynchronous, chunk-based model where data lies in remote storage is the management of object lifetimes, due to the overlapping nature of downloading and processing jobs.

Some JIT compiled operators such as Join or Aggregation build Hash-Tables, whose behavior is different for fixed-size types (e.g. Integers) vs. variable-size types such as Strings. For fixed-size types, a copy-by-value is performed, meaning the table owns the data; blocks can be safely released once they have been processed by the upstream operators. In contrast, for variable-size types, a shallow copy is performed (meaning only pointer and length) — which results in dangling pointers to garbage or newly downloaded chunk data once blocks are released by the Asynchronous Scheduler.

We therefore rely on thread-local String Heaps (using the pre-existing `TupleBuffer` class), where a deep copy is performed at the LLVM JIT layer (see `StringTy::createCopy` function in `Types.cc`). The original string data can then safely be freed along with the processed blocks. The `stringHeaps` are pre-allocated with 64 KiB as an initial capacity to reduce allocation overhead during LLVM pipeline execution. A few of the utilities in `Murmur.h` and `Hash.h` had to be adapted to handle memory-unaligned strings, since the `TupleBuffers` for the string heaps tightly pack the string data for efficiency. Finally, the post-scan-pipeline hash-table aggregation logic across all threads was slightly modified to correctly account for the heterogeneous distribution of the entries.

2.4 Parquet

For the remote data storage format, we opted for Parquet [3], which is designed specifically for large-scale analytical workloads on tabular data stored in blob storage like Amazon S3. It has been widely adopted by analytical engines such as DuckDB [17], Cloud Data Warehouses such as Snowflake [18] or Data Lakehouses such as Databricks [19].

2.4.1 Row-Group-Based Storage

The most compelling reason to use the Parquet format for this project was its underlying physical layout; in Parquet, data is stored in a columnar fashion within individual row groups, allowing for a smooth transition from the decompressed chunk format to the raw column pointers desired within the context of LLVM pipeline execution. The dataset is split at the top-level into row groups, each containing the columns for a set of entire rows, as shown in Figure 2.2. In this project, we appended a few steps to the classic TPCH Generator program utility suite to generate `.parquet` files from the created binaries. Important configurable parameters of the script include the number of rows per row group and the compression level (e.g. `uncompressed`, `SNAPPY` [20] or `ZSTD` [21]).

The compression level has a direct impact on the runtime of the decompression stage and therefore the overall runtime of jobs for threads assigned to `JobType::PROCESSING`, which is described in detail in Section 3.2. Compression level, number of rows per chunk, number of columns, and length of the variable-size data types (strings) ultimately determine the compressed chunk size, which ultimately determines the request size. In the AnyBlob paper, Durner, Leis, and Neumann suggest 8–16 MiB requests are cost-throughput optimal, which should be taken into account when generating the `.parquet` data files. Alternatively, a potential direction for further development of the engine could be the implementation of arbitrary-range requests in the scheduler logic described in Section 3.2.

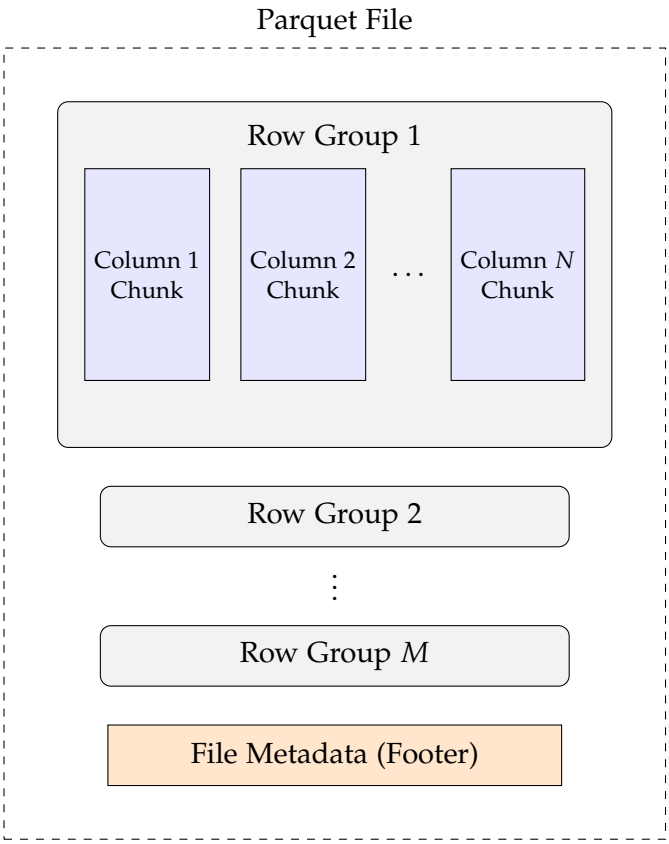


Figure 2.2: Parquet File Internal Structure.

2.4.2 Reading Data from Remote

The most straightforward solution for querying the Parquet data from remote would be to store the row groups for a table in a single `.parquet` file, since the format is self-describing: at the end of a `.parquet` exists a footer containing the metadata for the row groups, including their offsets within the file, allowing for ranged requests of specific groups. However, a slight incompatibility between the `AnyBlob` library and the format presented itself, namely the lack of support for HEAD requests. This makes it impossible to know exactly where the footer of a `.parquet` file lies without storing this information in advance. Assuming the existence of some kind of size fetching utility, the serial discovery process for the row group metadata would still involve multiple dependent round-trip requests, as shown in Figure 2.3.

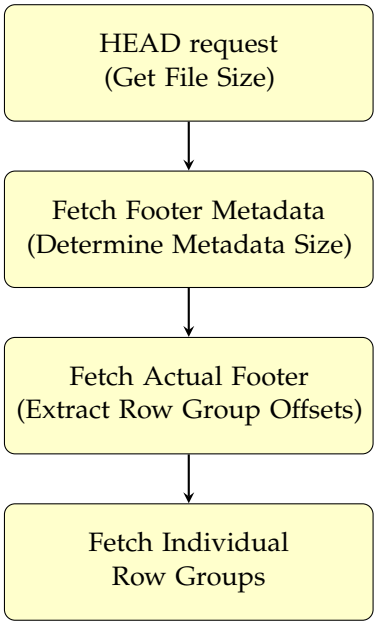


Figure 2.3: Standard Serial Discovery Process for Parquet Metadata on Remote Storage.

Additionally, a low-level Arrow reader for the compressed Parquet Data would need to be implemented; although it lies beyond the scope of this project, it is an interesting point for potential future extension of the engine. In the meantime however, we decided to store one Parquet file per row group, appending an index to the file name for ease of sequential processing such as `lineitem_1.parquet`, `lineitem_2.parquet`, etc.

This allows us to rely on the high-level `parquet::arrow::FileReader` class to read & decompress entire row groups as individual parquet files, without worrying about offsets of ranged requests or manual decoding of the present byte sequences (e.g. the identification of the used compression algorithm). The number of row groups (chunks, individual files) for any given table, along with the compressed size, uncompressed size and exact number of rows for each row group is stored in a separate JSON metadata file, fetched on startup of the query. The metadata discovery overhead is therefore limited to the schema information GET request and the individual table fetch requests, significantly reducing the initialization latency that would come with the classic Parquet discovery process.

2.5 Chunk Processing Implementation

2.5.1 Database and Tables

To replace the in-memory DB logic with a dynamic, metadata-reliant one, we implement a few classes: the `Database` class serves as the holder to the individual `ParquetTable` (unique) pointers. Each `ParquetTable` relies on a `ParquetFileMetadata` struct, which contains the necessary information for the Query Engine to process chunks, namely the `table_name` and a vector of `RowGroupInfo` structs. These structs individually hold their own ID with the respective remote object key (e.g. 12 -> `lineitem_12.parquet`), the uncompressed and compressed sizes of the row group, as well as the number of rows. The `ParquetTable` objects each hold a state-machine vector of `ChunkStates`, which track the current state of each individual row group within the table. Their values are defined as `UNLOADED` -> `PREPARING` -> `DOWNLOADING` -> `DOWNLOADED` -> `DECOMPRESSING` -> `DONE`. This status is updated after the successful completion of each logical step in the chunk fetch and decompression pipeline.

2.5.2 Chunks

For the actual chunk data, we distinguish between two structs: `FetchedChunk` and `DecompressedChunk`, for raw and ready-for-processing columns, respectively. The `FetchedChunk` only needs to track its index within the row group, size and raw bytes. The `DecompressedChunk` holds the index, number of rows, an `arrow::Table` `arrow_holder` variable, which contains the freshly decompressed parquet data columns, and a map of column names to raw pointers.

For variable-length data, it additionally contains `string_buffers` that own the memory for strings converted into the engine's internal dense string heap format, ensuring their lifetime matches that of the decompressed chunk. The pointers and buffers serve as an intermediate step between Parquet and LLVM and are passed to the LLVM kernel for execution. The compression type is determined at runtime by the `arrow::FileReader` from the raw chunk bytes. The `ParquetTable` object holds one current decompressed chunk at a time per thread, which the LLVM kernel processes.

3 Scheduling Implementation

For this chapter, we largely focus on Scan pipelines, since these serve as the entry point for data being fetched from remote into the engine. Due to the chunk-based nature of the Query Engine, different multi-threaded execution models are possible. We therefore implement the `QueryScheduler` base class, which holds the pointer to the `Database` struct and serves as parent class for `SyncParquetScheduler` (baseline), and the AnyBlob-inspired `AsyncParquetScheduler` class, which is the focus of this thesis.

3.1 Synchronous Baseline

For the baseline implementation, we take inspiration from the original P2C class `MultiThreadedScheduler`, which simply iterates over the in-memory data in `chunk_size` increments and executes the JIT-compiled kernels. The chunk indexing logic of the new `SyncParquetScheduler` remains mostly the same, each thread making use of an atomic counter `next_chunk_idx` to know which chunk must be fetched and processed.

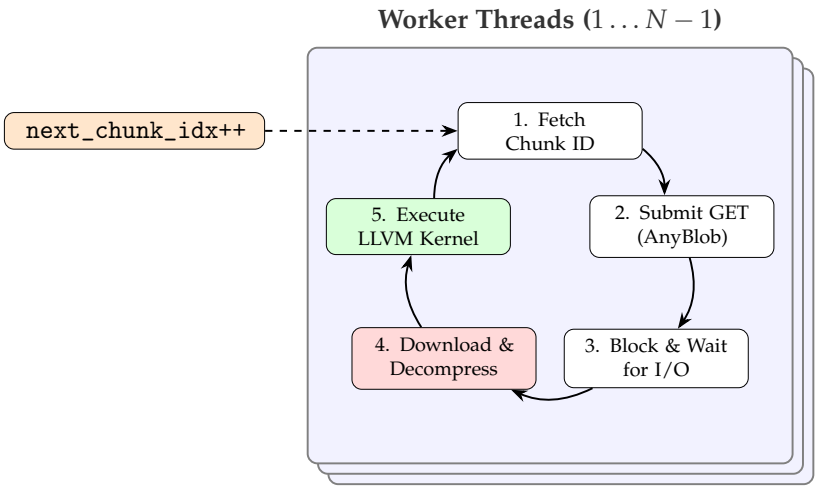


Figure 3.1: Synchronous Scheduler Execution.

Individual threads perform all of the necessary chunk-based operations sequentially: first, a synchronous (blocking) `GET` request is sent to fetch the raw compressed chunk through the `AnyBlob` library (for details, see `ParquetTable::fetchChunkSync()` function in `ParquetTable.cc`). Subsequently, it is decompressed and set as the current thread-local chunk in the table object; the pointer array for the JIT execution is built and the LLVM pipeline function `kernel_ptr` runs on the decompressed data.

3.2 Adaptive Asynchronous Scheduler

While the multi-threaded synchronous version can provide good execution results, as shown in our Benchmarking results in Chapter 4, it fails to fully adapt to the network environment, therefore not maximizing resource utilization. As Durner, Leis, and Neumann mention in their paper, threads are blocked a considerable amount of time waiting for I/O, when they could be working on other tasks, such as decompression or processing.

In this project, we closely follow the approach described in the AnyBlob paper, where the `io_uring`-based `AnyBlob` library utilities are leveraged to integrate efficient asynchronous object store retrieval into the RDBMS Umbra, in an effort to "saturate the available network bandwidth with low CPU resource consumption" [4]. In our case, we integrate `AnyBlob` with the high-performance P2C engine through the implementation of the `AsyncParquetScheduler`, which relies on aggregation of statistics to reassign thread jobs for maximum throughput.

3.2.1 Thread Jobs

We categorize the possible thread jobs as `PREPARATION`, `RETRIEVAL` and `PROCESSING`. Upon completion of an individual task, threads call the `pickJob()` function to get the new job. This way, the assignment is delegated to the worker threads themselves, which eliminates the single point of contention caused by an 'active' scheduler thread. Furthermore, a `TaskedSendReceiverHandle` is initialized for each thread to poll for I/O if they ever get assigned the retrieval job. The preparation job consists of a request to our `BlockPool` memory pool (described in Section 3.3.1) and a call to the non-blocking `anyblob::network::Transaction::processAsync()` function, which asynchronously registers the `GET` request with the `io_uring`-backed network stack. No data fetch is performed at this stage, the submitted requests need to be driven by threads assigned to `JobType::RETRIEVAL`.

For the retrieval job, the thread performs a `process()` call to its handle with the `oneQueueInvocation` parameter set to true, which instructs the handle to drain the queue and exit, rather than waiting for new submissions in daemon mode. This is crucial, since the scheduler thread would otherwise have to externally stop the handle for job reassignment. On completion, the callback places the newly fetched chunk in a lock-free decompression `ready_queue` (described in detail in Section 3.3.2), and stops the `io_handle` so the thread can be assigned a new job. In this case, we choose fine-grained adaptivity of the scheduling over reducing the function call overhead caused by repeatedly entering and exiting the `AnyBlob` network polling utilities. Our experiments reveal that attempting to bypass this via "batched" processing of queue submissions (not explicitly stopping the handle) can result in the thread being locked into the retrieval job longer than desired, hurting the performance.

The processing task comprises two coupled, compute-heavy stages: decompression through the `decompressChunk()` function and JIT-compiled kernel execution over the raw column pointers from the chunk. Figure 3.2 depicts the execution flow of the `AsyncParquetScheduler`.

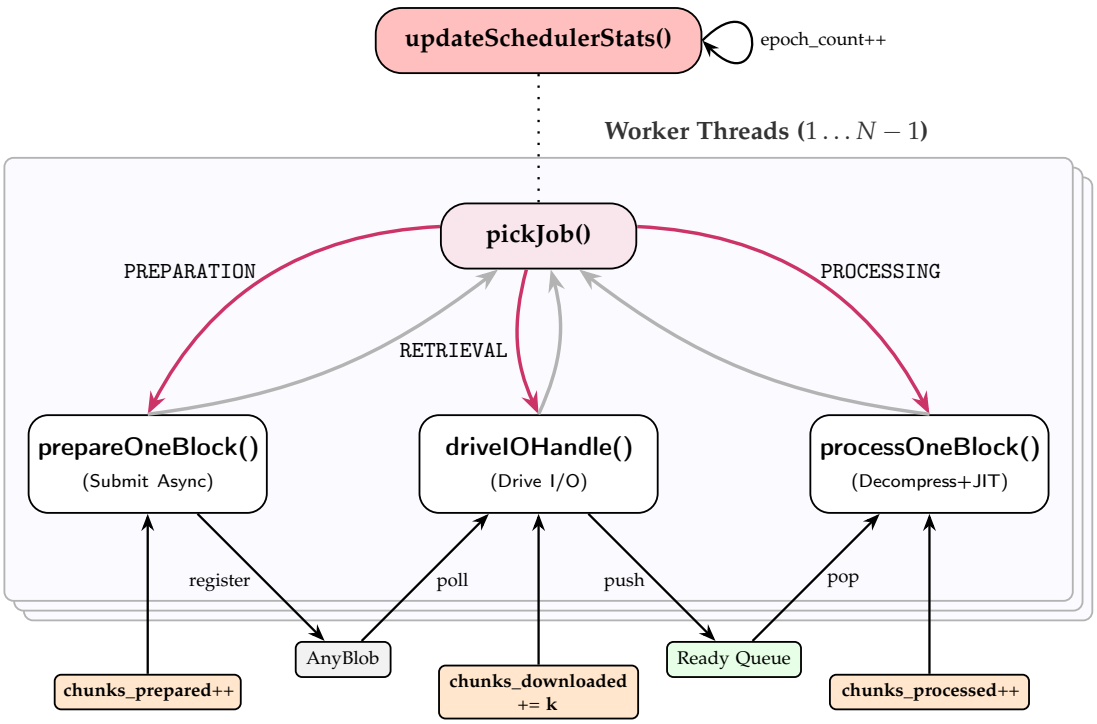


Figure 3.2: Asynchronous Scheduler Execution.

3.2.2 Statistics Aggregation

As described in the original AnyBlob paper, the AsyncParquetScheduler attempts to achieve a balance between processing and retrieval performance. We therefore rely on aggregation of performance statistics during execution (through `std::atomic` variables), allowing the Scheduler to make informed assignment decisions. Each processing thread measures its own execution time and counts the number of processed chunks, which we use to compute the mean processing throughput per thread.

The same occurs for the retriever threads, which measure time spent downloading data and number of raw chunks downloaded, allowing us to compute the network throughput. Additionally, we track outstanding bytes on submission of a chunk download request; this information (such as `compressed_chunk_size`) is easily accessible through the `ParquetFileMetadata::RowGroupInfo` fetched on startup.

3.2.3 Performance Adaptivity

As mentioned in the original paper, we should capitalize on the large available instance bandwidth capabilities without neglecting the balance between processing and retrieval performance, which could both easily become a bottleneck. Therefore, we strictly follow the principles described in section 4.3 of the paper ([4]) and implement a robust version of their Algorithm 1. Through the mechanisms described in this section, we ensure it is able to withstand fluctuating resource conditions, which are unfortunately frequent in Cloud Instances.

We rely on the main scheduling thread to take snapshots of the retrieved performance statistics in fixed time intervals (see Section 3.2.5), resetting the counters on epoch start. This thread stores target counts for preparation, retrieval and processing threads in atomic variables, which are read by the individual worker threads in the `pickJob()` function upon completion of their current task. Retrieval and processing speed are calculated based on the processed and downloaded bytes and time elapsed. It is worth noting that for the retrieval speed, we take the total epoch time as the denominator, whereas in the processing speed we divide by the exact time spent running the LLVM kernel. This distinction is necessary because retrieval speed is constrained by the physical wall-clock bandwidth of the network instance; the Network Interface Card (NIC) and `io_uring` kernel interface work asynchronously in the background, meaning bytes may start flowing into the kernel's buffers even before a thread enters `driveIOHandle()` to poll for them. Only counting the time spent polling would not be representative of the actual retrieval performance.

In contrast, processing speed must be normalized to the aggregate CPU time to accurately reflect the throughput of a single worker thread, independent of scheduling stalls or jitter. Absolute processing speed is then computed by aggregating per-thread speed over the number of processing threads, and an instant ratio of processing versus retrieval speed is computed, which we will subsequently correct (see Section 3.2.4). The corrected ratio is used to calculate the number of retriever threads necessary for balancing retrieval and processing performance, relative to the fixed maximum number of retriever threads.

In the AnyBlob paper, Durner, Leis, and Neumann rely on an elaborate heuristic (section 2.8 of the paper) to determine the necessary number of requests to saturate the network bandwidth; in a nutshell, combining these values with the number of concurrent requests per thread allows them to specify an upper bound for retriever threads (after which the returns become largely diminishing). Since we would like to keep the default value for concurrent requests per thread set internally by the `AnyBlob` library, and it being notably difficult to assess the true bandwidth of the cloud instance, we rely on a simpler method. In our implementation, we determine this number purely experimentally for our `c6in.16xlarge` instance, arriving at the conclusion that $\frac{1}{4}$ to $\frac{1}{2}$ of the total number of available threads yielded the best results, with the "sweet spot" around $\frac{1}{3}$ (see Section 4).

3.2.4 Stabilization of Adaptivity Algorithm

Due to the possibility of inconstant network and compute conditions on the Cloud Instances, we attempt to reduce scheduling oscillation through an Exponential Moving Average (EMA). We perform this operation on the input, meaning the measured statistics, to ensure that one misrepresentative epoch does not have a large negative impact on the scheduling decision. Additionally, we use EMA on the output, the new target retriever threads, to reduce systemic oscillation of the Scheduler. We set an EMA factor α for the input and β for the output, both configurable in `config.json`.

Algorithm 1: Function: updateEMA(currentAverage, instantValue, smoothingFactor)

1 $newAverage = smoothingFactor \times instantValue + (1 - smoothingFactor) \times currentAverage$;
2 **return** $newAverage$;

It is important to point out that we can set a valid starting value for the retriever threads on initialization, but the Scheduler does not have access to an initial ratio of processing versus retrieval speed without measurements. Providing an arbitrary initial value, such as 1, could lead to a 'cold-start' problem where early performance is heavily degraded by an incorrect assumption. Instead, we apply a bias-correction term (Algorithm 2) to the EMA, guaranteeing that early measurements are weighted correctly. We take inspiration from the Adam optimizer in Machine Learning [22], used for bias-correction of accumulated gradients in the parameter update steps. This ensures the algorithm remains robust even in the initial steps of execution.

Algorithm 2: Function: biasCorrection(smoothedValue)

1 $biasCorrection = 1 - (1 - \alpha)^{epochCount}$;
2 **return** $\frac{smoothedValue}{biasCorrection}$;

We also set a threshold on the difference between the current and the next target for retriever threads necessary for reassignment to take place, in an effort to reduce jitter due to small changes. Algorithm 3 shows the complete process of target updates with the applied stability modifications in detail.

Algorithm 3: Modified Adaptivity Algorithm.

1 $retrieveSpeed = \frac{retrievedBytes}{epochDuration}$;
2 $processSpeed =$
$(totalThreads - currentRetrieving - currentPreparing) \times \frac{processedBytes}{processingTime}$;
3 $instantRatio = \frac{processSpeed}{retrieveSpeed}$;
4 $smoothedRatio = updateEMA(instantRatio, smoothedRatio, \alpha)$;
5 $currentRatio = biasCorrection(smoothedRatio)$;
6 $requiredBandwidth = \min(bandwidth, bandwidth \times currentRatio)$;
7 $requiredRetrieverThreads = \min(totalThreads \times currentRatio, totalThreads)$;
8 **if** $|requiredRetrieverThreads - currentTarget| > (totalThreads \times \tau)$ **then**
9 $\quad nextTarget = updateEMA(requiredRetrieverThreads, currentTarget, \beta)$;
10 **else**
11 $\quad nextTarget = currentTarget$;
12 $nextTarget = \max(1, \min(nextTarget, totalThreads - 1))$;
13 $maxOutstanding = requiredBandwidth \times 2.0 \times epochDuration$;

As a final measure for early stabilization of the algorithm, we introduce a configurable warm-up time before the `updateSchedulerStats()` takes effect, to ensure enough statistics are retrieved for meaningful updates. This period amplifies the impact that the initial retriever thread number has on the overall scheduler's performance, due to the possibility of under- and overassignment; too few threads and the retrieval performance takes too long to ramp up, too many threads and the performance hit caused by threads inevitably finding an underfilled `io_uring` retrieval queue becomes noticeable. Through our experiments, we found $\frac{1}{2} \times$ `max_proportion_retrievers` to yield solid results on startup.

3.2.5 Scheduling Parameters

Since different Cloud Instances may have different magnitudes of condition fluctuations, it is highly useful to be able to configure the parameters of our Scheduler. We therefore rely on the `Config` class, which allows us to specify the following:

blockpool_size_gb : Total memory capacity allocated to the `BlockPool` memory pool (see Section 3.3.1).

max_network_bandwidth_gb : Physical upper bound of the instance's network interface (prevent the Scheduler from over-saturating the link).

max_prepared_mb : Cap on total volume of outstanding asynchronous chunk requests.

max_retriever_proportion : Maximum proportion of retriever threads relative to the total number of available threads.

epoch_duration_ms : Time interval between snapshots taken by the main thread to aggregate statistics and set targets.

warm_up_time_ms : Configurable delay at the start of a query execution before the adaptivity algorithm begins making reassignment decisions.

damping_factor (β) : Controls the smoothing of the output target to prevent the Scheduler from overshooting during thread reassignment.

ema_alpha (α) : Smoothing factor for EMA applied to the performance ratio of processing versus retrieval speed.

reassignment_threshold (τ) : Deadband value that prevents jittering by requiring a minimum difference before changing the target for retriever threads.

3.3 Optimization

While the adaptive nature of the scheduler ensures high resource utilization, achieving peak engine performance requires us to make an effort to eliminate low-level system inefficiencies. For this purpose, we employ a few distinct optimization techniques to reduce contention points in the processing stage and improve the overall performance of our Query Engine. We focus on two primary areas: kernel-mode overhead caused by the system memory allocator and stalling issues caused by locks in high-concurrency environments.

3.3.1 BlockPool

For any implementation of a High-Performance Query Engine, reducing the kernel overhead is crucial, and one of the most critical causes of latency due to excessive context switches is frequent memory allocation and deallocation. To mitigate this, we implement a size-classed memory pool (implemented in `BlockPool.h`), a common practice among modern systems such as Meta's Velox [23], which employs multiple memory pools in its sophisticated Memory Manager system [24].

The `BlockPool` is a singleton and acts as a specialized allocation proxy for buffers required during Parquet chunk downloading, decompression, and processing. It operates on the principle of resource reuse and manages a collection of pre-allocated memory segments (blocks) categorized by size classes, in an effort to reduce the frequency of `mmap` and `brk` system calls. When a worker thread requires memory for a newly retrieved chunk, it requests a block from the pool, which ensures size class alignment: small requests (under 4MB) are aligned to 512-byte boundaries, while larger requests are rounded up to the nearest power of two to optimize for modern CPU cache lines and prevent fragmentation. The pool provides an `acquire()` interface that returns an `std::unique_ptr` with a custom deleter. This ensures that when the processing of a chunk is complete, the memory is not returned to the OS via `free()` but is instead automatically recycled back into the pool's internal `size_classes` map for immediate reuse. It can be configured to use the efficient `jemalloc` [25] library as a backend, further reducing contention (while providing a fallback to `std::aligned_alloc`).

Our `BlockPool` is used across multiple stages of the query execution, namely for managing the buffers necessary for downloading raw Parquet chunk data, the buffers where the decompressed chunk data is stored and the buffers utilized by the LLVM kernel for the columnar data.

Lastly, a wrapper class was implemented for the `arrow::FileReader` class to rely on the memory pool backend during the decompression stage (implemented in `ArrowBlockPoolWrapper.h`). Due to the fact that the simple implementation of our pool relies on a `std::mutex` for the block tracking mechanism, we should mention that lock contention could become a concern in highly concurrent environments (such as instances with 128+ threads). However, in our experiments (instances with 16–64 threads), the performance increase is still present and justifies the use of `BlockPool` compared to relying exclusively on `jemalloc` (see Section 4.2.4).

3.3.2 Lock-Free Data Structures

Due to the high-concurrency environment where modern Query Engines run (often 64+ threads), lock contention can become a major concern for overall throughput. Consequently, it is crucial to minimize the usage of classic mutex-based synchronization logic. In our implementation, three critical paths require specialized lock-free synchronization. The first one is the `ready_queue`, responsible for block handover between the asynchronous downloading and decompression/processing threads.

To ensure that exactly one consumer thread acquires a given chunk without a lock, we rely on the `moodycamel::BlockingConcurrentQueue` [26]. This Multi-Producer Multi-Consumer (MPMC) queue avoids global lock contention by utilizing a sharded architecture: it maintains a collection of independent Single-Producer Multi-Consumer (SPMC) sub-queues for each producer thread. By decentralizing data storage and using atomic primitives for memory ordering instead of heavy-weight mutexes, it achieves near-linear scalability even in many-core environments. This primitive has been adopted by various performance-critical modern systems, most notably within the PyTorch Machine Learning framework [27].

Since threads assigned to processing may encounter an empty queue, we rely on the queue's `wait_dequeue_timed()` functionality to reduce the waste of CPU resources resulting from busy-spinning. By introducing a small wait timing (1 ms), we greatly reduce the costly "chunk_misses" that spin straight back into the `pickJob()` function. At the same time, we ensure that the threads don't wait too long and therefore hurt the adaptivity of the Scheduler.

4 Performance Evaluation

To test out the performance of our adaptive Scheduler, we use a Parquet TPCH dataset generated by an adapted `dbgen` script suite, which performs gradual chunk-wise generation of data and subsequent compression to `.parquet` format. Due to the fact that our engine assumes one `.parquet` file per chunk, we must generate the dataset taking this into account. To properly adhere to the morsel-driven-parallelism approach of the AnyBlob paper, which relies on the aforementioned chunk size range of 8 to 16 MiB, we generate chunks of 250,000 rows per chunk using `zstd` compression as the default. While this size range provides no issues for large datasets such as ours, it should likely be scaled down in the case of smaller datasets due to a large chunk to dataset size proportion. Sticking to a large number of morsels enables the granularity that our `AsyncParquetScheduler` needs for smooth updates; while we implement various stabilization mechanisms, bursts in processing or retrieval can still lead to significant hits to the overall performance.

Our main dataset has scale factor 100, stored on Amazon S3 [1], and for the engine we use two separate Amazon EC2 [28] instances: the smaller c6a.2xlarge for development purposes and the network-optimized `c6in.16xlarge` for the final benchmarking rounds (both x86_64 architecture). The specifications of this instance most relevant to our project are a network bandwidth link of 100 Gbit (12.5 GB), 32 physical cores (64 VCPUs) and 128 GiB Memory. Detailed instance specifications can be checked on the EC2 Instance Type Manual [29]. The S3 bucket and EC2 instances are allocated in the same region (`eu-north-1`) to reduce latency and data transfer costs.

4.1 Parameter Search

For the purpose of finding the optimal configuration of parameters for our scheduling algorithm, described in detail in Section 3.2.5, we use grid search. This is implemented in `p2c_bench.py`, where we rely on the internal aggregated performance statistics of the engine and `perf` measurements to get the best possible combination of parameters.

We order the aggregated `.csv` data by overall runtime and generate `FlameGraphs` for the top 5 combinations, allowing us to understand the bottlenecks that might be slowing down the query execution. For each parameter combination, along with the `SyncParquetScheduler`, we ran the `TPCH-Q01` query five times. This query was selected due to its status as a retrieval-heavy chokepoint (scanning the entirety of the large `lineitem` table) while performing dense arithmetic aggregations.

```sql
select
    l_returnflag,
    l_linestatus,
    sum(l_quantity) as sum_qty,
    sum(l_extendedprice) as sum_base_price,
    sum(l_extendedprice * (1 - l_discount)) as sum_disc_price,
    sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)) as sum_charge,
    avg(l_quantity) as avg_qty,
    avg(l_extendedprice) as avg_price,
    avg(l_discount) as avg_disc,
    count(*) as count_order
from
    lineitem
where
    l_shipdate <= date '1998-12-01' - interval '90' day
group by
    l_returnflag,
    l_linestatus
order by
    l_returnflag,
    l_linestatus;
```

Listing 4.1: Query 1: TPCH-Q01

Some of the configuration parameters have to be fixed for the scheduler to behave as intended. Due to the testing instance (`c6in.16xlarge`), we set the following parameters outside of our grid search:

blockpool_size_gb : 120 (due to EC2 instance)

max_network_bandwidth_gb : 12.5 (due to EC2 instance)

epoch_duration_ms : 50

warm_up_time_ms : 300

reassignment_threshold (τ) : 0.05

For the remaining parameters, we tested the following value combinations:

max_prepared_mb : [256, 512, 1024, 2048, 4096, 8192]

max_retriever_proportion : [0.125, 0.25, 0.33, 0.5]

damping_factor (β) : [0.3, 0.5]

ema_alpha (α) : [0.1, 0.2]

4.2 Benchmarking Results

From the aggregated results, we draw some conclusions regarding the parameter choice; in this section, we evaluate the impact each individual parameter has on overall runtime and runtime variance across runs. Due to the specified parameter lists, we are looking at 96 datapoints, each one constructed from the 5-run-average of that particular setup.

4.2.1 Irrelevant Parameters

We first look at `damping_factor` (β) and `ema_alpha` (α), the coefficients that determine the smoothing of processing versus retrieval ratio and target retrieval threads, respectively. Interestingly, the choice between the two given values for each one doesn't seem to have a large impact on the performance of our Scheduler. This can be ascertained due to the similarity of the boxplots modeling the variance of the runtime, depicted in Figure 4.1, although setting β to 0.5 and α to 0.1 seems to yield slightly better results. We therefore fix those values in subsequent plots to reduce noise in the visualization.

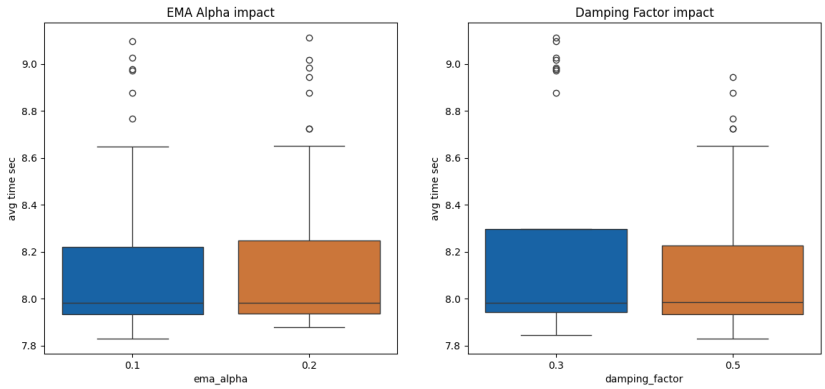


Figure 4.1: Boxplot of Parameters with Negligible Impact.

4.2.2 Runtime Averages and Variance

We closely look at the distribution of runtimes for each parameter combination, focusing on the variance (s^2). Due to the cloud-native nature of our Scheduler, we look for settings where the performance reveals little fluctuation, even if individual datapoints of variants with higher variance may show better absolute performance. We plot the correlation of each parameter in the search list to the runtime variance in Figure 4.2.

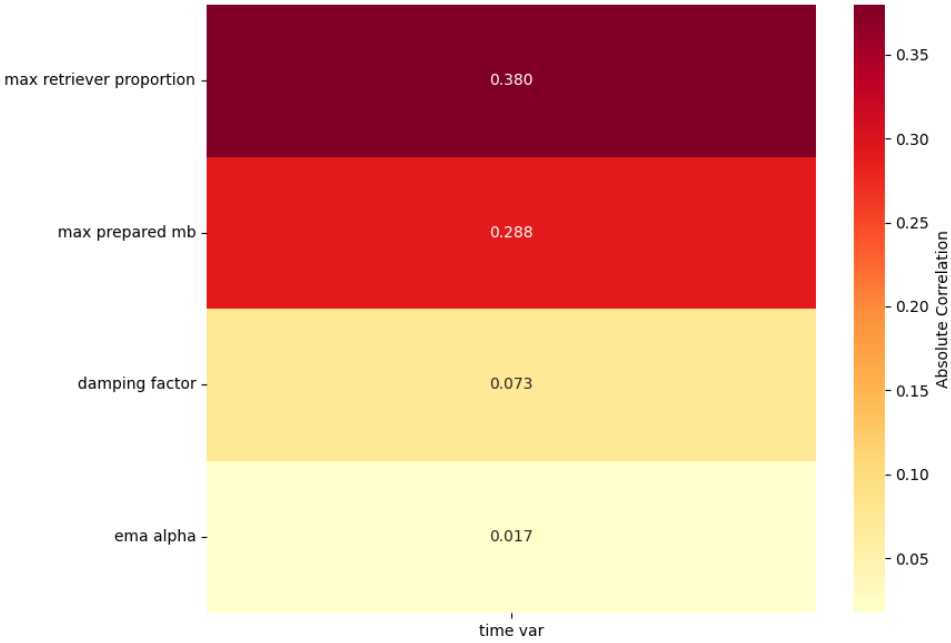


Figure 4.2: Absolute Correlation of Parameters with Runtime Variance.

This chart shows us that the largest (absolute) correlation lies with the maximum number of retriever threads and maximum amount of outstanding data (MB). As mentioned previously, the max number of retriever threads should have been calculated through the number of concurrent requests per thread, like in the original AnyBlob paper, in order to saturate the bandwidth. It is therefore expected that experimental determination of this parameter would induce high variance in the runtimes. Additionally, we confirm that α and β don't show a large impact, provided they lie within a reasonable range. Regarding the more impactful parameter choices, we conclude that a misconfigured proportion of maximum retriever threads or outstanding bandwidth cap can make the engine unproductive.

The proportion set to $\frac{1}{8}$ reveals the poorest runtimes, whereas at $\frac{1}{2}$ performance is slightly worse than the optimum on average; this leads us to believe the proportion of $\frac{1}{4}$ to $\frac{1}{3}$ to be the "sweet spot". Additionally, we conclude that a lower maximum amount of prepared megabytes results in lower average runtimes, as seen in Figures 4.3 and 4.4. However, very low thresholds (<512) can be highly detrimental to the algorithm's performance, consequently leading us to settle on 1024 MB.

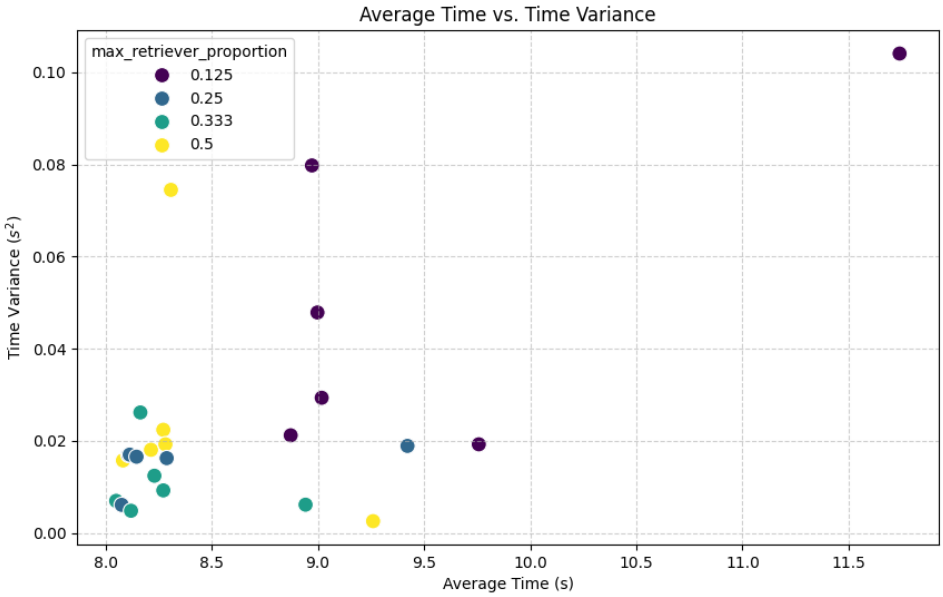


Figure 4.3: Scatterplot of Retriever Proportion Runtimes Distribution.

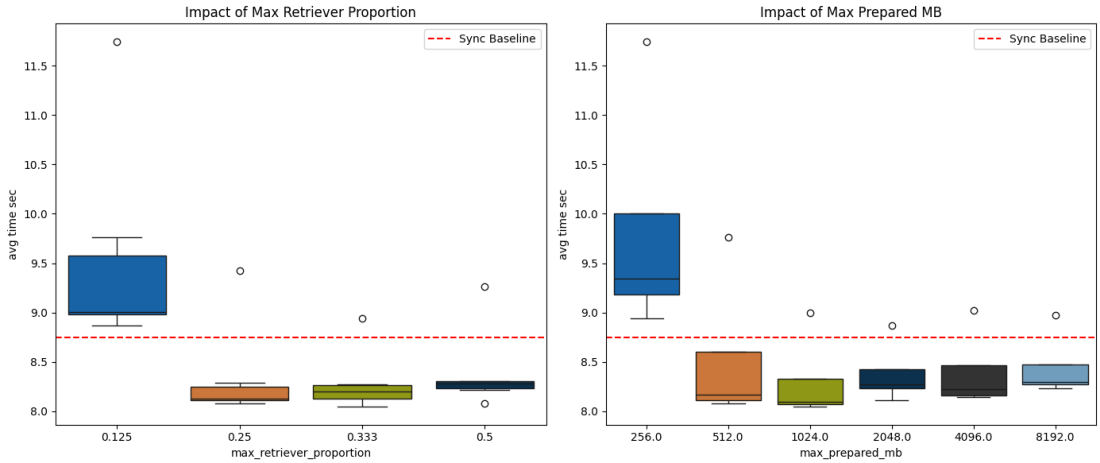


Figure 4.4: Boxplots of Retriever Proportion & Max Prepared MB Runtimes Distribution.

The impact of the outstanding MB cap can be explained upon inspection of the Scheduler's logs (in `Debug` mode). If set too high, a longer period of correction is necessary, hurting the overall performance. If set too low, the Scheduler might not be able to converge to the optimal retriever versus processor thread proportions.

4.2.3 Final Choice

The plot below shows the average runtime for each parameter combination (with fixed α and β). Taking the reduced variance into account, we choose the following as the definitive parameter setting for our `c6in.16xlarge` instance: α **= 0.1,** β **= 0.5,** `max_retriever_proportion` **= 0.33,** `max_prepared_mb` **= 1024**. The most crucial conclusion to be drawn is probably the fact that a poorly tuned asynchronous Scheduler performs considerably worse than the "naive" synchronous baseline.

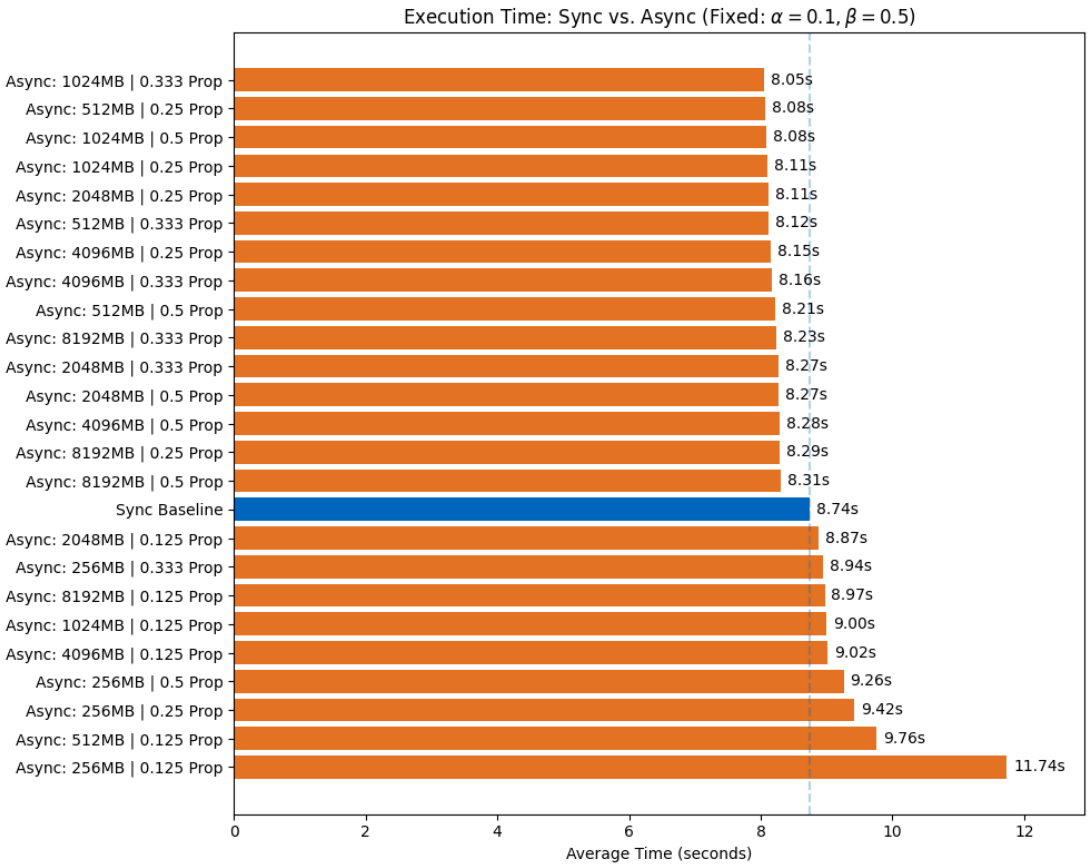


Figure 4.5: Average Runtime of Each Parameter Combination.

4.2.4 Scheduler Profiling

In an ideal execution environment, a balanced query such as `TPCH-Q01` would achieve a near-total overlap between retrieval and computation. Under these conditions, the query runtime would be limited solely by the time required to saturate the physical network link. To establish a theoretical performance baseline, we can perform a "back-of-the-envelope" calculation based on the hardware constraints of our environment: scanning the compressed `lineitem` table (approximately 19 GB) over a 100 Gbit/s network link should, in theory, result in a retrieval time of roughly 1.52 seconds. However, our experimental results largely deviate from this ideal limit, suggesting that physical bandwidth is not the sole arbiter of performance. The following analysis aims to understand execution gaps and identify where the system encounters its bottlenecks. For the best performing parameter combinations, we use the Linux profiling tool `perf` to obtain insights on the execution. We take our best parameter combination (specified in Section 4.2.3) and analyze the `perf` data through a `FlameGraph`. It becomes clear that the `libparquet` utilities, which include decompression, take up 59% of the samples, followed by the `sendReceive()` function at around 17%. This is recognizable from the large yellow chunks labeled `libparquet.so` on the FlameGraph in Figure 4.6, indicating that the system becomes compute-bound during the transformation of Parquet's columnar format into the engine's internal representation.

To better gauge the overhead caused by the decompression in isolation, we run the `TPCH-Q01` benchmark on an uncompressed Parquet dataset of scale factor 100, averaging 5 runs once again. We rely on our grid search approach a second time to allow for different parameter combinations, since the proportion of compute versus network workload is shifted dramatically: the lineitem table now consists of just over 4000 chunks of about 9.2 MB on average, totaling around 37 GB — almost double the previous dataset size. Counter-intuitively, the query runtimes are considerably slower, as seen in Table 4.1, which shows the 3 best performing parameter combinations.

Table 4.1: Performance of Scheduler on Uncompressed Parquet Dataset.

Version	Max Retriever Proportion	Max Prepared (MB)	Avg. Runtime (s)	Runtime Variance
sync	—	—	11.664	1.971
async	0.5	1024	9.688	0.010
async	0.5	2048	9.794	0.021
async	0.5	4096	9.946	0.023

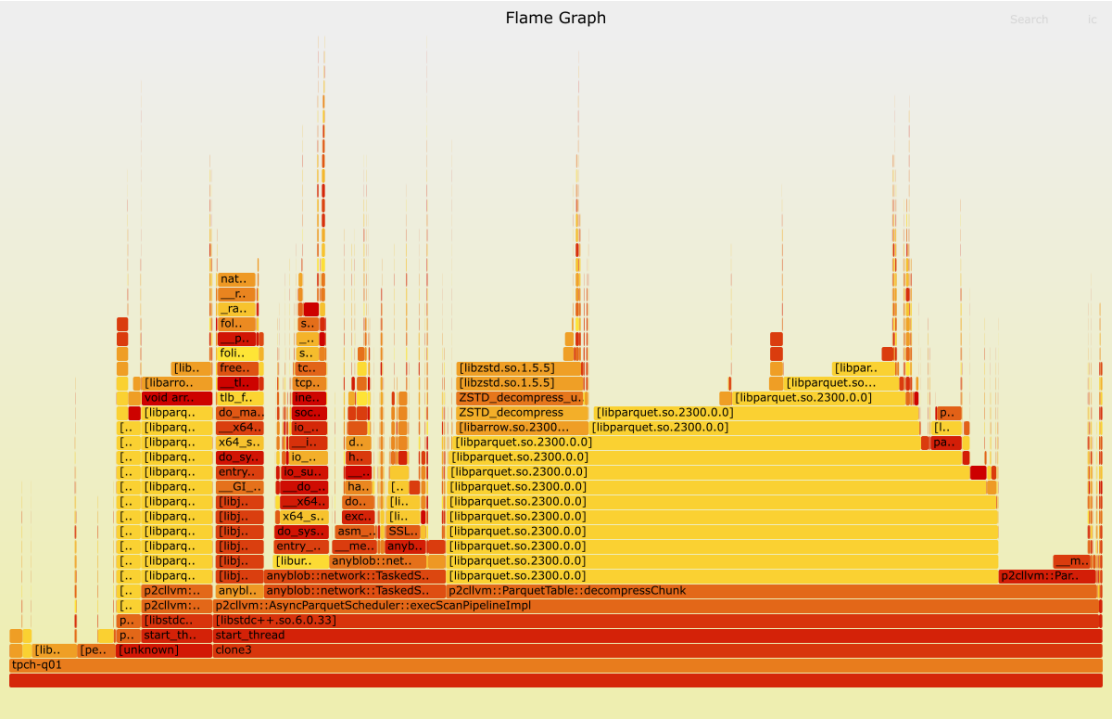
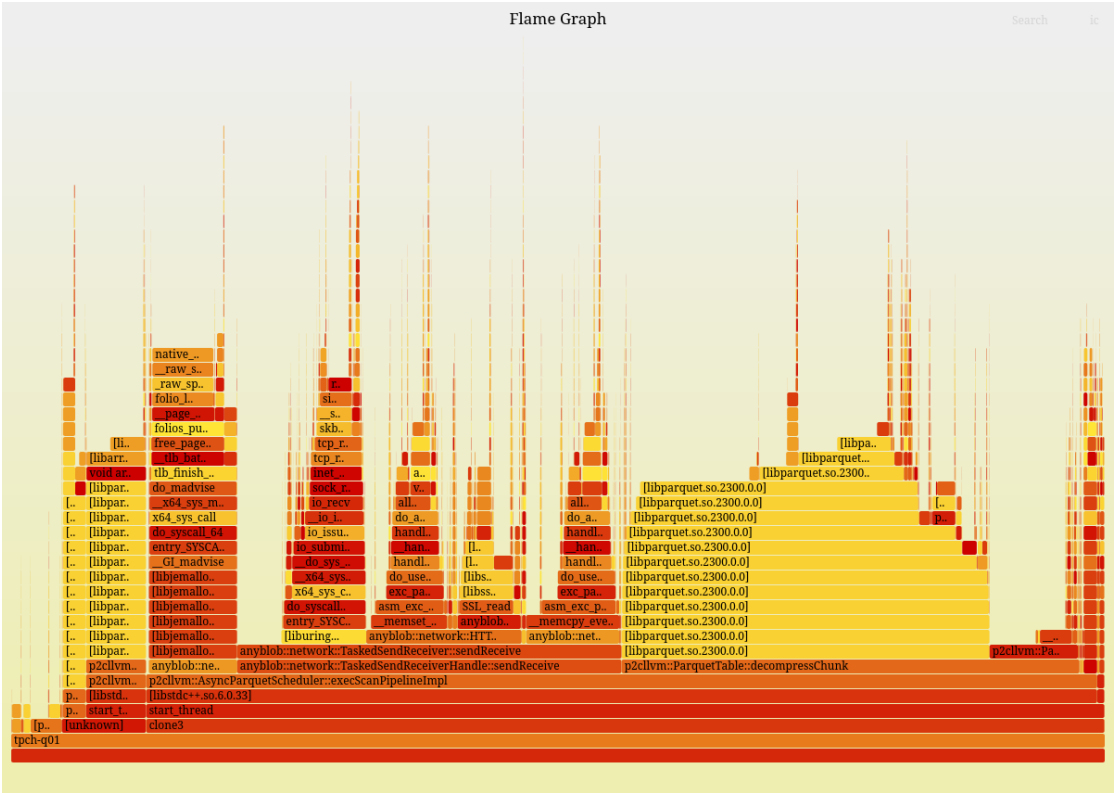


Figure 4.6: Perf FlameGraph of Scheduler on Compressed Dataset.

Upon inspection of the recorded `perf` data for the best performing parameter setting, we verify that the percentage of samples taken in retrieval increase considerably to just over 35%. Nevertheless, the overhead caused by the `arrow/libparquet` reader is still massive, making up over 41% of the samples, which leads us to believe the `arrow` reader to lag behind in performance compared to the rest of the program.


Flame Graph
Search
ic
native_..
__raw_s..
_raw_sp..
folio_l..
__page_..
folios_pu..
free_page..
__tlb_bat..
tlb_finish_..
do_madvise
__x64_sys_m..
x64_sys_call
do_syscall_64
entry_SYSCA..
__GI_madvise
[libjemallo..
[libjemallo..
[libjemallo..

Subsequently, we inspect the difference between our `BlockPool`-reliant implementation and the version that utilizes only the `jemalloc` library. In both cases, we profile the query execution over the compressed dataset, as there are more frequent memory operations. We observe that the memory allocation overhead, represented by the flame columns on the right in Figure 4.9, takes up a larger chunk of the sample space in the `jemalloc` version. Blocks labeled with function signatures such as `_memcpy_evex_unaligned_erms` or `dallocx` become more accentuated, compared to our optimized version. We compare some well-performing parameter combinations ($\alpha = 0.1$, $\beta = 0.5$, `max_prepared_mb` = {512, 1024}, `max_retriever_proportion` = {0.25, 0.33}) and conclude that the `jemalloc`-reliant implementation is around 18% worse on average.

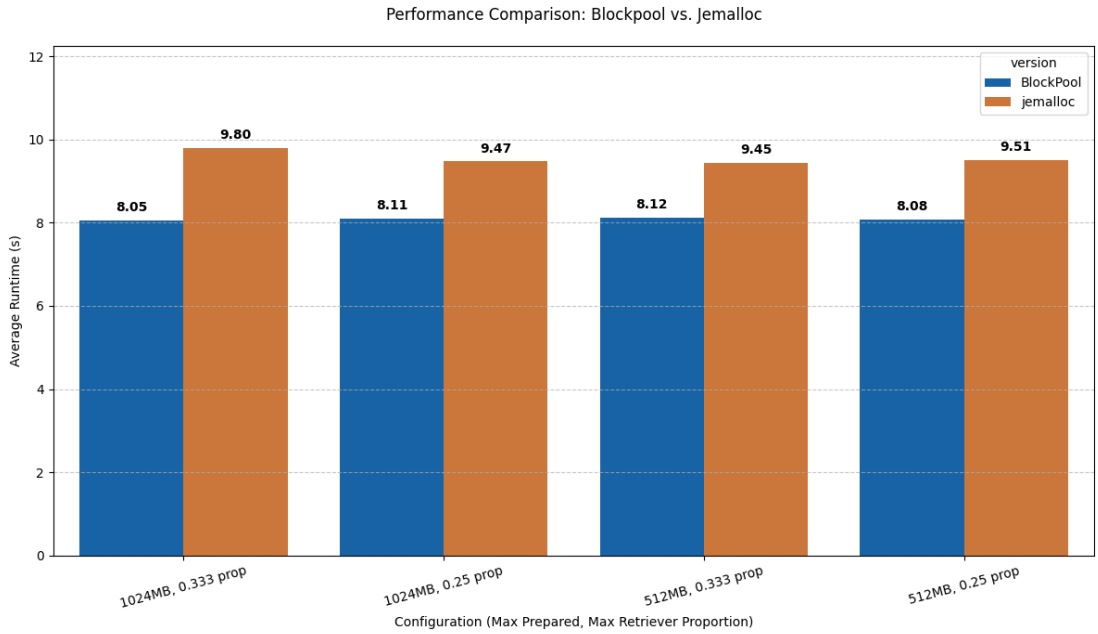


Figure 4.8: BlockPool vs. jemalloc Comparison.

A more thorough inspection of the recorded CPU metrics reveals the overhead caused by the kernel-reliant memory management of the `jemalloc`-only implementation. Table 4.2 shows selected performance metrics per tuple of our TPCH dataset. The average amount of cycles, kernel cycles and L1 cache misses increase significantly in the `jemalloc`-only version, and a slight increase in instructions and Last-Level Cache (LLC) misses is also visible.

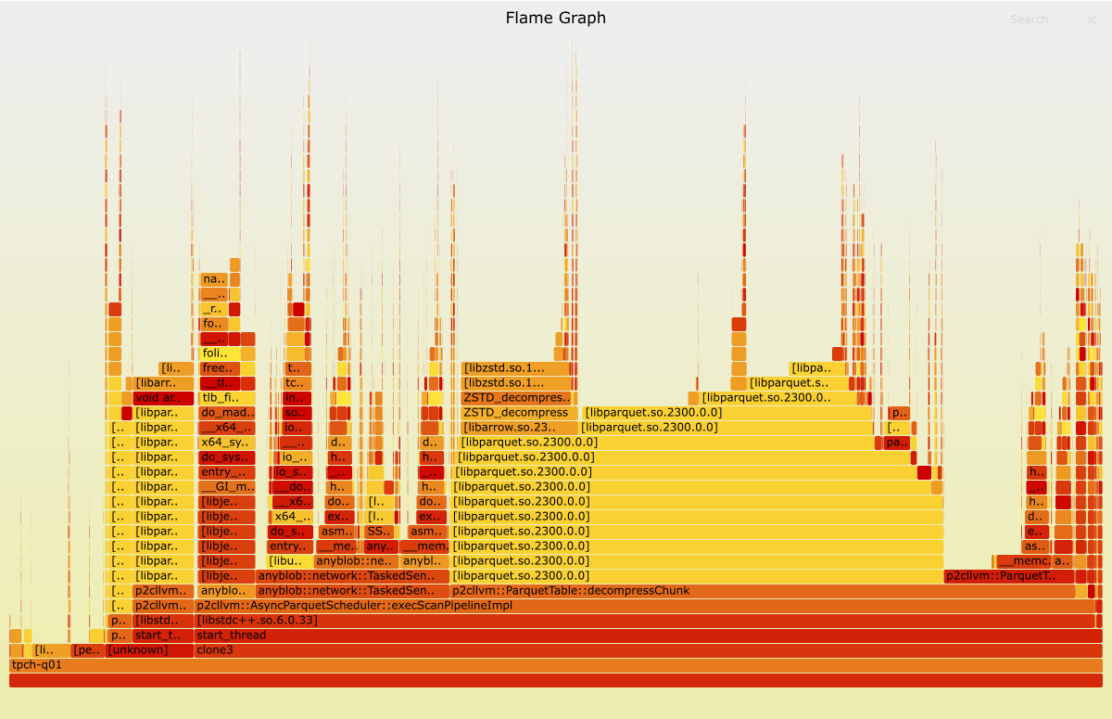


Figure 4.9: Perf FlameGraph of jemalloc-only Scheduler.

Table 4.2: CPU Metrics of BlockPool vs. jemalloc Scheduler.

Version	Avg. Cycles	Avg. Kernel Cycles	Avg. Instructions	Avg. L1 Misses	Avg. LLC Misses
BlockPool	2195.72	409.59	2600.35	25.51	9.39
jemalloc	2463.76	699.28	2785.10	27.71	9.42

4.2.5 DuckDB Comparison

To evaluate the performance of our adaptive scheduler against a state-of-the-art baseline, we compare our results with DuckDB [17], widely considered one of the industry standards for in-process analytical query execution and cloud-native data lake exploration. As a final benchmark, we run the `TPCH-Q01` query on DuckDB (once again averaged out across five runs, cold start each time), deployed on the same `c6in.16xlarge` instance. It is crucial to mention that a second scale-factor 100 dataset was uploaded to our S3 bucket with the standard one `.parquet` file per table, generated with `zstd` compression through the DuckDB `dbgen()` function. This is due to the fact that DuckDB does not support our ad-hoc solution of one `.parquet` file per row group. Due to the size of the dataset, the impact caused by the initial discovery logic should not be significant, compared to our implementation.

Interestingly, we conclude that our `AsyncParquetScheduler` outperforms DuckDB by about 48%. The performance disparity stems from DuckDB's reliance on a vectorized execution model, which may be efficient for general workloads but introduces materialization overhead between operators. It also limits the potential for deep pipeline fusion of our JIT-compiled approach. Furthermore, DuckDB utilizes a generic filesystem abstraction layer as interface to S3, reliant on `HTTPFS` [30]; this adds significant middleware overhead and interpretation logic compared to our specialized `AnyBlobManager`.

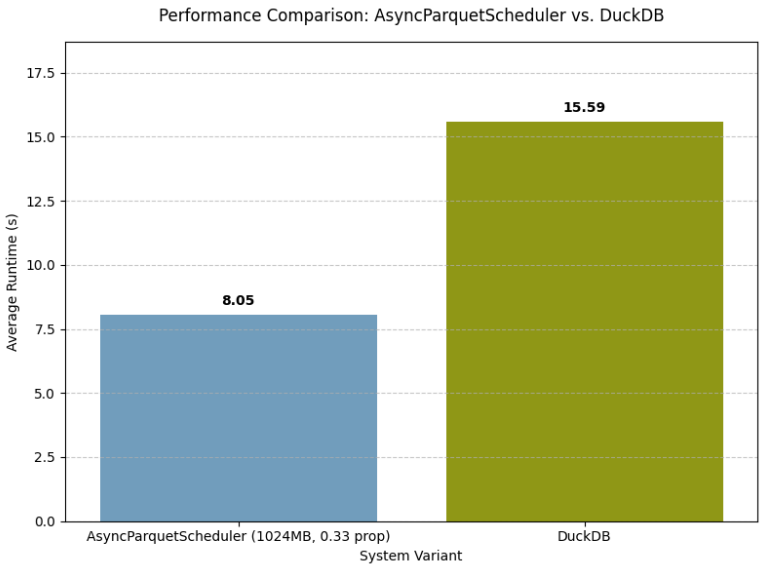


Figure 4.10: TPCH-Q01 (SF100, ZSTD Compression): AsyncParquetScheduler vs. DuckDB Performance Comparison.

5 Conclusion & Future Work

5.1 Conclusion

This thesis shows how JIT-compilation-based query engines can be performant in cloud-native settings, demonstrated through the transition of the in-memory P2C prototype to a dynamic analytical engine supporting remote stores. By integrating the `AnyBlob` retrieval library and implementing an adaptive scheduling mechanism, we were able to efficiently overlap network latency and computation. Our findings highlight that while asynchronous I/O can be deeply beneficial, the stability of the scheduling algorithm and the specific tuning of parameters is what ultimately allows the system to saturate the high-bandwidth network links without starving the CPU of processing tasks.

5.2 Future Work

While the current implementation provides a robust foundation for cloud-based query execution, several possibilities for future research and optimization remain:

Decoupling Decompression and Processing. Currently, the `JobType::PROCESSING` task bundles Parquet decompression with LLVM kernel execution. Due to the use of the high-overhead Parquet reading & decompressing utilities, decoupling the job into separately scheduled subtasks would allow the engine to scale decompression threads independently of execution threads, further refining resource utilization.

Metadata-Driven Pruning. In cloud-native warehouses like Snowflake [7], performance is significantly enhanced by "data skipping", where the engine inspects `parquet` file footers or external metadata to skip entire row groups that do not satisfy query predicates. Currently, P2C fetches data based on the full range of a dataset. Future work could implement a pre-fetch metadata analysis phase that leverages min/max statistics for more selective range requests. Thereby, the engine would reduce the total volume of data transferred, leading to significant cost and latency savings in large-scale analytical workflows.

Integrated Parquet Reader. To simplify deployment and reduce initial discovery overhead, the engine currently relies on a "one-file-per-row-group" strategy. A significant future improvement would be the implementation of a native reader capable of performing `HEAD` requests (once `AnyBlob` supports it) and ranged-fetches. This would allow the engine to query standard, single-file `.parquet` datasets without pre-processing and enable the aforementioned Metadata-Driven Pruning.

Alternative Storage Formats. As established in Section 4.2.4, the abstraction layers of `parquet::arrow` utilities introduce significant computational overhead. Future research should explore the integration of storage formats with higher read & decompression throughput to alleviate this CPU bottleneck. A primary candidate is BtrBlocks [31], a columnar storage format format specifically optimized for data lakes. By utilizing lightweight encoding schemes, BtrBlocks could potentially reduce the CPU cycles spent on data ingestion, allowing the P2C engine to maintain its high-performance compiled execution path even at 100 Gbit/s and beyond.

Multi-Query Concurrency. The current `AsyncParquetScheduler` is optimized for the execution of single queries. In a production environment, multiple queries often run concurrently; future work should explore how the adaptivity algorithm handles resource contention across multi-query contexts and whether a global rather than a per-query Scheduler is more effective.

Cache Management. Although the `BlockPool` optimizes memory reuse, there is no caching logic across queries. This leads us to believe that a persistent local NVMe cache layer could be integrated, which would mitigate overall network-latency-induced overhead by prioritizing local "warm" data rather than querying from remote each time.

Memory Pool Management. Our `BlockPool` demonstrated performance gains in experimental trials, but it is far from perfect; in a higher-concurrency production environment, it would be susceptible to mutex contention, and a hierarchical memory management structure such as the one from Velox [24] would likely need to be implemented.

Abbreviations

P2C Plan-to-C

TUM Technical University of Munich

NVMe Non-Volatile Memory Express

DIS Decentralized Information Systems and Data Management

JIT Just-In-Time

NIC Network Interface Card

EMA Exponential Moving Average

MPMC Multi-Producer Multi-Consumer

SPMC Single-Producer Multi-Consumer

LLC Last-Level Cache

List of Figures

2.1 LLVM JIT execution flow 3
2.2 Parquet File Internal Structure. 6
2.3 Standard Serial Discovery Process for Parquet Metadata on Remote Storage. 7

3.1 Synchronous Scheduler Execution. 10
3.2 Asynchronous Scheduler Execution. 12

4.1 Boxplot of Parameters with Negligible Impact. 21
4.2 Absolute Correlation of Parameters with Runtime Variance. 22
4.3 Scatterplot of Retriever Proportion Runtimes Distribution. 23
4.4 Boxplots of Retriever Proportion & Max Prepared MB Runtimes Distribution. 24
4.5 Average Runtime of Each Parameter Combination. 25
4.6 Perf FlameGraph of Scheduler on Compressed Dataset. 27
4.7 Perf FlameGraph of Scheduler on Uncompressed Dataset. 28
4.8 BlockPool vs. jemalloc Comparison. 29
4.9 Perf FlameGraph of jemalloc-only Scheduler. 30
4.10 TPCH-Q01: AsyncParquetScheduler vs. DuckDB Performance Comparison 32

List of Tables

4.1 Performance of Scheduler on Uncompressed Parquet Dataset. 26
4.2 CPU Metrics of BlockPool vs. jemalloc Scheduler. 31

Bibliography

[1] Amazon Web Services. *Amazon Simple Storage Service (S3)*. 2026. URL: https://aws.amazon.com/s3/.

[2] LLVM Foundation. *LLVM Compiler Infrastructure*. 2026. URL: https://llvm.org/.

[3] Apache Software Foundation. *Apache Parquet*. 2026. URL: https://parquet.apache.org/.

[4] D. Durner, V. Leis, and T. Neumann. "AnyBlob: Exploiting Cloud Object Storage for High-Performance Analytics." In: *Proceedings of the VLDB Endowment (PVLDB)* (2023). URL: https://www.vldb.org/pvldb/vol16/p2769-durner.pdf.

[5] Microsoft Corporation. *Microsoft Azure*. 2026. URL: https://azure.microsoft.com/en-us.

[6] Google LLC. *Google Cloud Storage*. 2026. URL: https://cloud.google.com/storage.

[7] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, and P. Unterbrunner. "The Snowflake Elastic Data Warehouse." In: *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*. 2016. URL: https://dl.acm.org/doi/epdf/10.1145/2882903.2903741.

[8] M. Vuppalapati, J. Mesterharm, R. Liu, L. Lin, Y. Zhang, C. Iorgulescu, L. Ryzhyk, A. Agarwal, A. Ashari, and C. Burns. "Building an Elastic Query Engine on Disaggregated Storage." In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 2020. URL: https://www.usenix.org/system/files/nsdi20-paper-vuppalapati.pdf.

[9] N. Armenatzoglou, S. Basu, N. Bhanoori, M. Cai, N. Chainani, K. Chinta, V. Govindaraju, T. J. Green, M. Gupta, S. Hillig, E. Hotinger, Y. Leshinksy, J. Liang, M. McCreedy, F. Nagel, I. Pandis, P. Parchas, R. Pathak, O. Polychroniou, F. Rahman, G. Saxena, G. Soundararajan, S. Subramanian, and D. Terry. "Amazon Redshift Re-invented." In: *Proceedings of the 2022 ACM SIGMOD International Conference on Management of Data*. 2022. URL: https://www.cs.cmu.edu/~15721-f24/papers/Redshift_Revinvented.pdf.

[10] P. Edara, M. Pasumansky, B. S. Reed, J. Shute, D. Agrawal, S. Jain, S. Kumar, S. Shrinivasan, A. Srivastava, et al. "When Metadata is Big Data: Google BigQuery's Modern Metadata Management System." In: *Proceedings of the VLDB Endowment (PVLDB)* (2021). URL: https://www.vldb.org/pvldb/vol14/p3083-edara.pdf.

[11] T. Steinert, M. Kuschewski, and V. Leis. "Cloudspecs: Cloud Hardware Evolution Through the Looking Glass." In: *Proceedings of the VLDB Endowment (PVLDB)* (2026). URL: https://www.vldb.org/cidrdb/papers/2026/p17-steinert.pdf.

[12] Technische Universität München - Fakultät für Informatik: Lehrstuhl III: Datenbanksysteme. *The Umbra Database System*. 2024. URL: https://umbra-db.com/.

[13] T. Neumann. "Efficiently Compiling Efficient Query Plans for Modern Hardware." In: *Proceedings of the VLDB Endowment (PVLDB)* (2011). URL: http://www.vldb.org/pvldb/vol4/p539-neumann.pdf.

[14] G. Graefe. "Volcano - An Extensible and Parallel Query Evaluation System." In: *IEEE Trans. Knowl. Data Eng.* (1994). URL: https://cs-people.bu.edu/mathan/reading-groups/papers-classics/volcano.pdf.

[15] V. Leis, P. Boncz, A. Kemper, and T. Neumann. "Morsel-Driven Parallelism: A Self-Aware Framework for Dynamic Multi-Core Query Execution." In: *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 2014. URL: https://db.in.tum.de/~leis/papers/morsels.pdf.

[16] Transaction Processing Performance Council. *TPC Benchmark H (Decision Support) Standard Specification*. Transaction Processing Performance Council. 2026. URL: https://www.tpc.org/tpch/.

[17] DuckDB Foundation. *DuckDB*. 2026. URL: https://duckdb.org/.

[18] Snowflake Inc. *Snowflake*. 2026. URL: https://www.snowflake.com/.

[19] Databricks Inc. *Databricks*. 2026. URL: https://www.databricks.com/.

[20] Google Inc. *Snappy: A fast compressor/decompressor*. 2024. URL: https://github.com/google/snappy.

[21] Y. Collet and Meta Platforms, Inc. *Zstandard: Real-time data compression algorithm*. 2025. URL: https://facebook.github.io/zstd/.

[22] D. P. Kingma and J. Ba. "Adam: A method for stochastic optimization." In: *arXiv preprint arXiv:1412.6980* (2014).

[23] Meta Platforms. *Velox*. 2023. URL: https://engineering.fb.com/2023/03/09/open-source/velox-open-source-execution-engine/.

[24] Meta Platforms. *Velox Memory Manager*. 2023. URL: https://facebookincubator.github.io/velox/develop/memory.html.

[25] J. Evans. *jemalloc: A scalable concurrent malloc implementation*. 2011. URL: https://jemalloc.net.

[26] C. Desrochers. *ConcurrentQueue: A Fast General Purpose Lock-Free Queue for C++*. 2014. URL: https://moodycamel.com/blog/2014/a-fast-general-purpose-lock-free-queue-for-c++.

[27] The Linux Foundation. *PyTorch: Optimized Tensor Library for Deep Learning*. 2026. URL: https://pytorch.org/.

[28] Amazon Web Services. *Amazon Elastic Compute Cloud (EC2)*. 2026. URL: https://aws.amazon.com/ec2/.

[29] Amazon Web Services. *Amazon EC2 Instance Types*. 2026. URL: https://aws.amazon.com/ec2/instance-types/.

[30] DuckDB Foundation. *HTTPFS*. 2025. URL: https://duckdb.org/docs/stable/core_extensions/httpfs/overview.

[31] M. Kuschewski, D. Sauerwein, A. Alhomssi, and V. Leis. "BtrBlocks: Efficient Columnar Compression for Data Lakes." In: *Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data*. 2023. URL: https://www.cs.cit.tum.de/fileadmin/w00cfj/dis/papers/btrblocks.pdf.